

Question 1 - Ma contribution

j'ai fait les fichiers

- DiscountCalculator.php
- EmailService.php
- SmsClient.php
- analyticsClient.php
- ClientService.php

j'ai fait le Ticket#106 par rapport de se qui a était demander

Question 2 - Une décision de conception

Le problème initial du projet c'est qu'il a était développer rapidement même si le code fonctionner, avec mon bînome nous avons eu la solution de faire du refactoring pour que le code sois plsu clean et réutilisable pour la suite

Cependant nous avons eu aussi la solution de la stratégie et du développement, le bute était de livrer le projet avant le D-Line

Mais nous avons garder la solution du refactoring car cela était plus important que de rendre un projet pas clean et pas spécialement compréhensible pour des futures review

Question 3 - SOLID et Design Patterns

Le projet combine le pattern Adapter et l'Inversion des dépendances BookingService dépend du contrat PaymentGateway et non des implémentations exactes StripeAdapter / PayFastAdapter. Cela permet de changer de fournisseur de paiement sans modifier le code métier.

Ce DIP reste partiel car Stripe est instancié par défaut directement dans le service, ce qui maintient un couplage fort.

Question 4 - Un test important

Le test crucial est « VIP + 3jour... » dans le fichier tests/characterization.php. Il prévient les régressions en validant l'ordre strict des réductions : les 10 % VIP doivent s'appliquer avant les 20 € du forfait 3 jours.

Pour 100 €, le résultat attendu est bien 70 €, soit $100 \times 0,90 - 20$, et non 72 € si l'ordre était inversé.

Question 5 - Review de votre propre projet

À conserver : Le contrat `PaymentGateway` et ses adaptateurs (`StripeAdapter`, `PayFastAdapter`). Ils découplent `BookingService` des API de paiement, rendant le système facilement évolutif.

À améliorer :

- Injection de dépendances : Les fournir à `BookingService` et `ClientService` au lieu de les instancier directement, pour faciliter les tests et le remplacement de services.
- Typage des montants : Abandonner les `float` pour éviter les erreurs de précision financière.

Abstractions (in)utiles : Aucune abstraction superflue n'est présente, les choix actuels sont justifiés. Il faut d'ailleurs conserver cette simplicité : l'actuel `DiscountCalculator` suffit amplement, inutile de complexifier le code avec un pattern `Strategy` pour les quelques remises existantes.

Question 6 - Évolution individuelle

Pour intégrer cette fonctionnalité, je créerais une classe `SlackClient` dans le dossier `src/Services/`, calquée sur le modèle de `SmsClient`, puis j'ajouterais son appel directement au sein de `ClientService`, `service()` afin de transmettre les détails de la réservation. Il faudrait ensuite mettre à jour le fichier `bootstrap.php` pour assurer le chargement de la nouvelle classe, ainsi que `tests/ClientServiceTest.php` pour valider son bon fonctionnement.